PNP测距

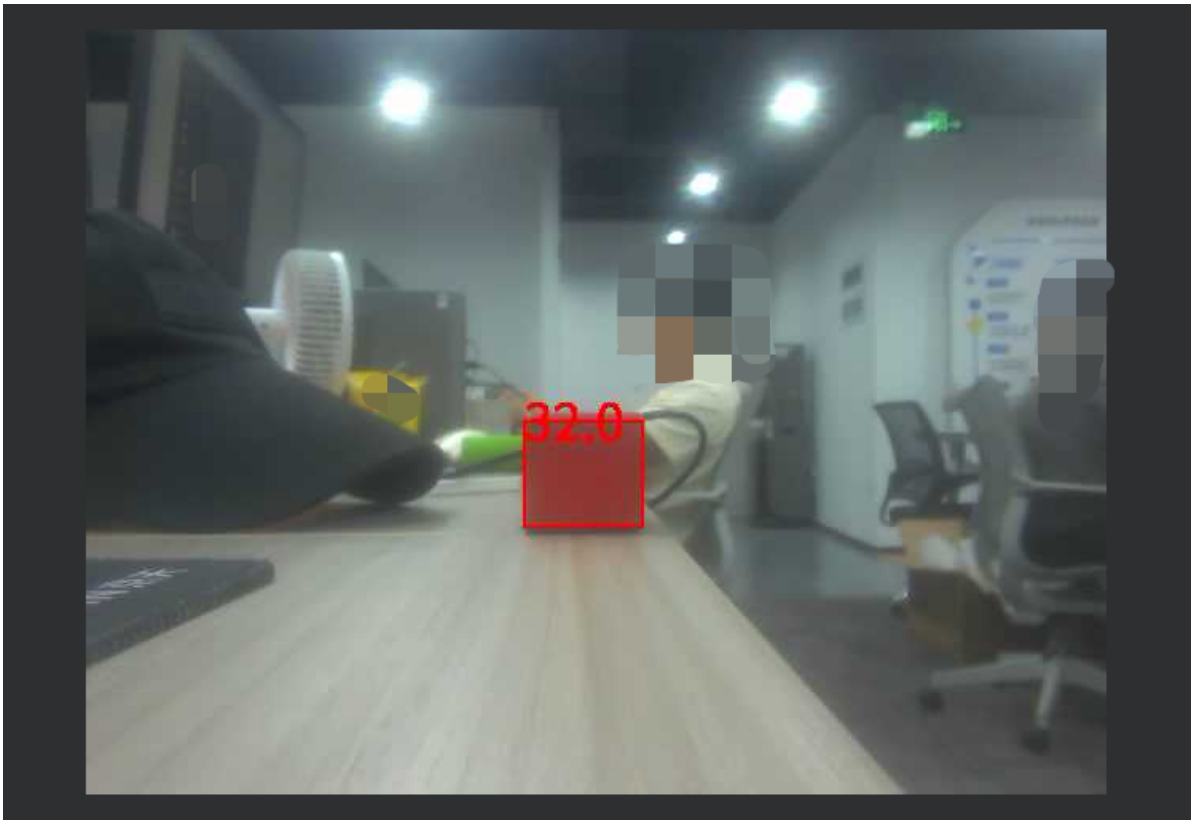
PNP测距
- 例程效果
- 原理讲解
  - 什么是图像彩色块检测与距离估计
- 代码概述
  - 导入模块
    - 设置图像尺寸
    - 初始化摄像头（RGB888 格式）
    - 初始化显示模块
    - 初始化媒体管理器并启动摄像头
    - 设置色块检测参数
    - 设置相机内参和畸变系数
    - 设置帧率计时器
    - 图像处理与彩色块检测及距离估计
    - 资源释放
  - 参数调整说明

例程效果

运行本节例程代码 [源码汇总/17.cv_lite/23.rgb888_find_blobs_and_distance.py]

本节我们将结合 `cv_lite` 扩展模块在嵌入式设备上实现 RGB888 格式的图像彩色块检测，并使用 PnP 算法估算检测到的色块距离。

> 实测距离为32.5cm，代码中有对距离做整除操作，如果希望提高精度，可以考虑修改代码的整数除法 (//) 为带小数的除法 (/)



原理讲解

什么是图像彩色块检测与距离估计

图像彩色块检测是一种计算机视觉技术，用于识别图像中特定颜色的区域（称为色块），并通过进一步计算估算这些色块在真实世界中的距离。彩色块检测基于颜色阈值分割图像，而距离估计则利用相机标定参数和物体实际尺寸来计算深度信息。

- **操作原理**：
  - **彩色块检测**：首先通过预定义的颜色阈值（例如 RGB 范围）对图像进行分割，提取符合条件的像素区域。随后，使用形态学操作（如腐蚀和膨胀）来优化检测结果，过滤噪声并连接相邻区域。检测结果通常返回色块的边界框（x, y, 宽度, 高度）。
  - **距离估计**：基于 PnP（Perspective-n-Point）算法，利用相机内参矩阵（camera_matrix）和畸变系数（dist_coeffs）来估计物体在三维空间中的位置。结合已知的物体实际尺寸（如色块的真实宽度和高度），通过三角测量计算距离。公式涉及投影几何和逆透视变换。
- **效果**：该操作可以实时检测图像中的特定颜色物体，并在图像上绘制边界框并显示估算距离，适用于需要快速定位和量测的场景。检测精度受颜色阈值、图像质量和相机标定影响。
- **应用场景**：彩色块检测和距离估计常用于机器人视觉（如跟踪彩色标志）、增强现实（AR）、工业自动化（如检测产品缺陷）和无人机导航等领域。在嵌入式设备上，这种方法结合了简单性和实用性，适合资源受限的环境。

在实际应用中，参数调整（如颜色阈值和最小面积）可以优化检测准确性，而相机参数确保距离估算的可靠性。

代码概述

导入模块

```
import time, os, sys, gc
from machine import Pin
from media.sensor import *    # 摄像头接口 / Camera interface
from media.display import *   # 显示接口 / Display interface
from media.media import *     # 媒体管理器 / Media manager
import _thread
import cv_lite                # 自定义计算机视觉模块 / Custom CV extension
import ulab.numpy as np       # MicroPython 版 NumPy / Lightweight NumPy for
uPython
```

设置图像尺寸

```
image_shape = [480, 640]  # 图像高 x 宽 / Height x Width
```

定义图像的分辨率为 480x640（高度 x 宽度），后续会根据此尺寸初始化摄像头和显示模块。

初始化摄像头（RGB888 格式）

```
sensor = Sensor(id=2, width=1280, height=960, fps=90)
sensor.reset()
sensor.set_framesize(width=image_shape[1], height=image_shape[0])
sensor.set_pixformat(Sensor.RGB888)  # 设置为 RGB888 格式 / Set pixel format to
RGB888
```

- 初始化摄像头，设置分辨率为 1280x960，帧率为 90 FPS。

- 将输出帧大小调整为 640x480，并设置为 RGB888 格式（三通道彩色图像，适合处理彩色块检测）。

初始化显示模块

```
Display.init(Display.VIRT, width=image_shape[1], height=image_shape[0],
to_ide=True, quality=50)
```

初始化显示模块，使用虚拟显示驱动（Display.VIRT），分辨率与图像尺寸一致。`to_ide=True` 表示图像会同时传输到 IDE 进行虚拟显示，`quality=50` 设置图像传输质量。

初始化媒体管理器并启动摄像头

```
MediaManager.init()
sensor.run()  # 启动摄像头采集 / Start capturing
```

初始化媒体资源管理器，并启动摄像头开始捕获图像流。

设置色块检测参数

```
threshold = [123, 145, 48, 79, 44, 73]  # 颜色阈值 [Rmin, Rmax, Gmin, Gmax, Bmin,
Bmax]
min_area = 10                          # 最小检测面积 / Minimum valid blob area
kernel_size = 1                         # 腐蚀膨胀核大小 / Morphological kernel size
```

- `threshold`：RGB 颜色阈值数组，用于定义检测的颜色范围。
- `min_area`：最小色块面积阈值，用于过滤过小的检测区域。
- `kernel_size`：形态学操作的核大小，用于优化检测结果。

设置相机内参和畸变系数

> 这里给的内参和畸变系数是亚博智能K230的原装摄像头GC2093，在1280*960下初始化对应的参数

```
camera_matrix = [
    1199.1495245491617, 0.0, 670.4213222250077,
    0.0, 1200.6879329729338, 475.11122810359876,
    0.0, 0.0, 1.0
]
dist_coeffs = [-0.0331624688297151, 0.04145795463721313, 0.0016964134879770974,
0.0008834973490863845, -0.05414957483933673]
dist_len = len(dist_coeffs)

# 实际 ROI 尺寸（单位：厘米）/ Real-world size of detected ROI (cm)
roi_width_real = 3.7     # 例如：色块宽 3cm / Blob width in real world
roi_height_real = 3.7    # 例如：色块高 2.8cm / Blob height in real world
```

- `camera_matrix`：相机内参矩阵，用于距离估算。
- `dist_coeffs`：相机畸变系数，用于校正镜头畸变。
- `roi_width_real` 和 `roi_height_real`：检测色块的实际物理尺寸，用于计算距离。

设置帧率计时器

```
clock = time.clock()
```

初始化帧率计时器，用于计算和显示每秒处理的帧数（FPS）。

图像处理与彩色块检测及距离估计

```
while True:
    clock.tick()  # 启动帧计时器 / Start FPS timer

    img = sensor.snapshot()          # 拍摄一帧 / Capture one frame
    img_np = img.to_numpy_ref()      # 获取图像 NumPy 引用 / Get NumPy reference to
RGB data

    # 色块检测：返回多个色块 [x, y, w, h, ...] / Detect color blobs
    blobs = cv_lite.rgb888_find_blobs(image_shape, img_np, threshold, min_area,
kernel_size)

    if len(blobs) > 0:
        # 获取第一个色块 ROI（矩形）/ Get first blob's bounding box
        roi = [blobs[0], blobs[1], blobs[2], blobs[3]]

        # 使用 PnP 估算色块距离 / Estimate distance via PnP
        distance = cv_lite.rgb888_pnp_distance(
            image_shape, img_np, roi,
            camera_matrix, dist_coeffs, dist_len,
            roi_width_real, roi_height_real
        ) // 2

        # 绘制矩形与距离文字 / Draw bounding box and distance text
        img.draw_rectangle(roi[0], roi[1], roi[2], roi[3], color=(255, 0, 0),
thickness=2)
        img.draw_string_advanced(roi[0], roi[1] - 20, 32, str(distance), color=
(255, 0, 0))

    # 显示图像到 IDE / Display image to IDE
    Display.show_image(img)

    # 打印当前帧率 / Print current FPS
    print("findblobs:", clock.fps())

    # 内存回收 / Garbage collection
    gc.collect()
```

- **图像捕获**：通过 `sensor.snapshot()` 获取一帧图像，并通过 `to_numpy_ref()` 将其转换为 NumPy 数组引用。
- **彩色块检测处理**：调用 `cv_lite.rgb888_find_blobs()` 函数检测色块，返回色块列表。
- **距离估计处理**：对检测到的色块，使用 `cv_lite.rgb888_pnp_distance()` 函数计算距离。
- **绘制和显示**：在图像上绘制色块边界框和距离文本，并显示到屏幕或 IDE 虚拟窗口。
- **内存管理和帧率输出**：调用 `gc.collect()` 清理内存，并通过 `clock.fps()` 打印当前帧率。

资源释放

```
sensor.stop()
Display.deinit()
os.exitpoint(os.EXITPOINT_ENABLE_SLEEP)
time.sleep_ms(100)
MediaManager.deinit()
```

参数调整说明

- `threshold`：RGB 颜色阈值数组 [Rmin, Rmax, Gmin, Gmax, Bmin, Bmax]，用于定义检测的颜色范围。调整值以匹配目标颜色（例如，根据环境光调整阈值），建议使用图像分析工具测试阈值范围。
- `min_area`：最小色块面积，值越大（如 10-50）过滤更多小噪声，值越小检测更敏感；根据图像分辨率和目标大小调整。
- `kernel_size`：形态学核大小，值越大（如 1-5）增强形态学操作效果，但可能导致色块边缘模糊；建议从 1 开始测试。
- `roi_width_real` **和** `roi_height_real`：色块的实际物理尺寸（单位：厘米），基于真实物体测量调整，以提高距离估算准确性。
- `camera_matrix` **和** `dist_coeffs`：相机标定参数，仅在相机变化时调整；确保使用校准工具获取精确值，以避免距离估算错误。